

Lab 4: Regular Expressions

MATH 153 • Data Wrangling and Engineering

Moussa Doumbia, Ph.D.

DATA	lab04_server_log.txt	lab04_products.csv
SUBMIT	lastname_lab04.ipynb, run top to bottom, plus a PDF export	
GRADING	100 points, with a 10-point bonus challenge • about 2.5 hours	

What this lab is about

You have a web-server log and a product catalogue. Neither is a tidy table, and neither can be loaded with `read_csv` and used as-is. Both are the ordinary situation in data engineering: the structure is there, but it is written into the *text* rather than into the schema.

A regular expression is how you get that structure out. This lab is not about memorising metacharacters. It is about three habits:

1. **Validate with `fullmatch`, extract with named groups.** Most regex bugs in production are one function call away from being impossible.
2. **Count what your pattern rejected.** A pattern that parses 90% of a file has told you something about the other 10%. Silence is not success.
3. **Know when to stop.** Some structure is not regular, and some patterns are a performance hazard. Recognising both is part of the skill.

Learning outcomes

On completing this lab you should be able to:

- (a) Choose correctly among `match`, `fullmatch`, `search`, `findall` and `finditer`.
- (b) Write a validating pattern for a documented grammar and report what it rejects.
- (c) Parse a semi-structured log into a typed `DataFrame` using named groups.
- (d) Use `re.sub` and the pandas `.str` accessors on a whole column.
- (e) Identify a pattern that is loose, ambiguous, or a backtracking hazard — and say why.

Setup

Put the two data files in the same folder as your notebook. Begin with this cell exactly as written:

```
import re
import numpy as np
import pandas as pd

LOG      = open("lab04_server_log.txt", encoding="utf-8").read().splitlines()
products = pd.read_csv("lab04_products.csv", dtype=str, keep_default_na=False)

print(len(LOG), "log lines |", products.shape, "product records")
```

Do not remove `dtype=str, keep_default_na=False`

These two arguments are what let you *see* the mess. Without them, pandas converts blanks and a dozen other tokens to `NaN` before you have looked at the file, and it infers types that hide exactly the defects this lab asks you to find. Every answer below assumes the file was read as text.

Contents

What this lab is about	1
Setup	1
1 Anatomy of a match	15 pts 4
2 Validation against a grammar	20 pts 4
3 Extraction: parsing the server log	30 pts 5
4 Substitution and splitting	15 pts 6
5 Regex in pandas	10 pts 7
6 Engineering judgment	10 pts 7
Challenge (bonus)	8
What to submit	8
Appendix: quick reference	10

1. Anatomy of a match

15 pts

Q1.1 `match`, `fullmatch`, `search`

[4 pts]

Search the string "HTTP/1.1 404 Not Found" for the pattern `\d{3}` using all three functions. Print all three results, then explain in two or three sentences how they differ.

Then run `re.match(r"\d{3}", "404abc")` and say why that result matters when you are validating data.

Q1.2 `findall` versus `finditer`

[4 pts]

Extract every three-digit number from the first ten log lines, once with `findall` and once with `finditer`. Show that the two return different *kinds* of thing, and state one situation in which you would need `finditer`.

Then predict, before running it, what `re.findall(r"(\d)(\d)(\d)", "a 404 b 500")` returns. Run it. If you were wrong, say what you expected and why.

Q1.3 Groups

[4 pts]

From the string "GET /catalog/item?id=1042 HTTP/1.1", extract the method and the path in a single match using **named** groups. Print `group(0)`, the method by number, the path by name, and `groupdict()`.

In one sentence: why are named groups worth the extra characters when you are about to build a DataFrame?

Q1.4 Raw strings

[3 pts]

Compare `repr(r"\b")` with `repr("\b")`. Then run `re.findall("\bcat\b", "cat concat cats")` — without the `r` — and explain the result.

What full credit looks like

Not “raw strings are safer.” Show the failure. A pattern that silently stops working, with no exception raised, is the thing you are learning to fear.

2. Validation against a grammar

20 pts

Validation asks a yes-or-no question about a *whole* value, so everything in this part uses `fullmatch` rather than `match`.

Q2.1 The SKU grammar

[7 pts]

A well-formed SKU is: two capital letters, a hyphen, exactly four digits, a hyphen, two capital letters. For example, AB-1042-XY.

Write the pattern. Report how many of the 60 records violate it, and print the distinct offending values.

Checkpoint You should find **48 valid** and **12 invalid**, spanning **two** different kinds of defect.

Q2.2 Email and phone

[7 pts]

Classify every `contact` value as **email**, **phone**, **blank**, or **neither**. Report the four counts.

The file contains five different phone notations. Write *one* pattern that accepts all five, and use `re.VERBOSE` so a reader can check each clause against the rule it encodes. In your write-up, list the notations you chose to accept.

Checkpoint 18 email, 30 phone, 6 blank, 6 neither. There is exactly **one** distinct “neither” value — say what is wrong with it.

Q2.3 Why not `re.IGNORECASE`?

[6 pts]

Re-run your Q2.1 check with `re.IGNORECASE` added. Report the new counts.

Then answer in a short paragraph: does the flag *fix* anything? If a colleague argued “lowercase SKUs are fine, just add `re.I`”, what would you say — and what would you implement instead?

3. Extraction: parsing the server log

30 pts

A log line looks like this (one line, wrapped here for the page):

```
203.0.113.19 - - [11/Mar/2026:08:11:00 -0500] "POST /api/v2/price HTTP/1.1"
200 27783 "-" "curl/8.4.0"
```

The fields, in order: client IP, two unused identity fields (always -), a bracketed timestamp, a quoted request line, the status code, the response size, a quoted referer, and a quoted user-agent.

Q3.1 One pattern for a log line

[12 pts]

Write a **single compiled pattern** with named groups `ip`, `ts`, `method`, `path`, `status`, `size`, `referer`, `agent`. Use `re.VERBOSE` and comment it. Report how many of the 117 lines parse.

Two things that will cost you points

The dot. HTTP/1.1 written without escaping the dot will match HTTP/1x1. In a grammar this strict, an unescaped `.` is a bug even when nothing in the file exploits it.

The size field. Not every line has a byte count. Look at what a HEAD request records before you decide the field is `\d+`.

Checkpoint 112 lines parse; 5 do not. If you got 90, re-read the note above.

Q3.2 The malformed lines

[6 pts]

Print the lines your pattern rejected and describe, in one sentence each, what is wrong with them. There are five, and no two fail for the same reason.

Q3.3 From matches to a DataFrame

[7 pts]

Build a tidy DataFrame from the parsed matches, with:

- `status` as an integer;
- `bytes` as a float, with the missing-body rows as `NaN` — **not** 0;
- `when` as a real datetime, parsed from the Apache stamp *including* its -0500 offset.

Checkpoint 112 rows. **22** of them have no response body.

In one sentence: why must the missing byte counts be `NaN` rather than `0`?

Q3.4 Three questions

[5 pts]

Using your frame, report:

- (a) the number of 4xx and of 5xx responses;
- (b) the five most-requested paths;
- (c) how many requests came from a client whose address is **not** IPv4.

A trap in part (c)

It is tempting to write a pattern for an IPv4 address and then keep only the rows that match. Do not. Use the pattern to **classify**, never to quietly discard — and say in your write-up how many rows a filtering approach would have lost.

Second question, worth thinking about even though it is not graded: does your IPv4 pattern accept 999.999.999.999? Should it?

4. Substitution and splitting

15 pts

Q4.1 Normalise the dimensions

[6 pts]

The `dims` column is written several ways: `12x8x3 in`, `12 x 8 x 3 in`, `9x9x9in`, `30.5cm x 20cm x 7.6cm`. Extract the three numbers and the unit into four columns.

Note that the unit may appear after any of the three numbers, or only after the last one. Your pattern must handle all of these.

Checkpoint 54 of 60 parse. Explain the other 6.

`re.IGNORECASE` is *appropriate* here, and was not in Q2.3. Say why the difference.

Q4.2 Pull the weight out of prose

[5 pts]

The `description` column contains a weight in most rows. Extract the value and the unit.

Be careful: some descriptions also mention a shelf size (`fits a 12in shelf`). A pattern that grabs the first number in the string will get the wrong one on several rows — anchor on something.

Checkpoint 42 of 60 rows contain a weight, in **two** different units.

One sentence: are the 42 extracted numbers comparable to each other? If not, what would you do?

Q4.3 `re.sub` with a function

[4 pts]

Redact the last four digits of every phone number in `contact`, leaving the rest of the value intact, so that (202) 555-0143 becomes (202) 555-XXXX.

Use a **function** as the replacement argument rather than a string. In one sentence, say what a function can do that a string replacement cannot.

5. Regex in pandas

10 pts

Q5.1 `str.extract` versus `str.extractall`

[5 pts]

Run both on the `dims` column with a pattern capturing a number. Report the shape of each result and explain the difference in one or two sentences.

What is the index of the `extractall` result, and why is it shaped that way?

Q5.2 The nullable-boolean trap

[5 pts]

Run this and explain the output:

```
s = pd.Series(["AB-1042-XY", "ab-6001-yz", None])
ok = s.str.fullmatch(r"[A-Z]{2}-\d{4}-[A-Z]{2}")

print(list(ok), ok.dtype)
print(list(~ok))
print(list(s[~ok]))
```

Which row disappears, and why? What is the one-method fix, and what does applying it commit you to believing about missing values?

6. Engineering judgment

10 pts

Written answers. Short is fine; vague is not.

Q6.1 `\S+` or `[^"]*` inside a quoted field?

[4 pts]

Take your Q3.1 pattern and change the `path` group from `[^"]*` to `\S+` (or the reverse, if that is what you wrote). Re-run it and compare the number of lines that parse.

One of the two versions parses *more* lines. Print the line that only the looser version accepts, and print the `path` it captured from that line.

The question

The looser pattern gave the higher parse count. Explain, in a short paragraph, why that is the **worse** result — and state the general rule about matching inside a delimited field.

Q6.2 Greedy versus lazy

[3 pts]

On the string `'"GET /a" and "POST /b"'`, compare `".*"`, `".*?"` and `"[^"]*"`. Report what each matches.

Two of them give the same answer. Which would you put in a parser, and why?

Q6.3 When regex is the wrong tool

[3 pts]

Time the pattern `(a+)+$` against the strings `"a"*n + "!"` for $n = 14 \dots 22$. Plot or tabulate the times.

Then answer: what is happening, why is it dangerous if the input comes from a user, and how would you rewrite the pattern? Name one other situation — not this one — where a regular expression is the wrong tool and say what you would use instead.

Challenge

bonus, 10 pts

Using your parsed request frame, produce **one** finding about this server that a colleague could act on — and state its denominator.

What earns the bonus

Not the statistic. The denominator sentence.

Your frame has 112 rows and the file had 117 lines. Any number you report is a number about 112 requests, and the 5 excluded lines are not a random sample — they are the lines where the log writer itself failed, which is exactly where a fault would surface first. A finding that does not say what it is a proportion *of* will not receive the bonus.

What to submit

1. `lastname_lab04.ipynb`, run top to bottom with **Kernel** → **Restart & Run All** before saving, with all outputs visible.
2. A PDF export of the executed notebook.
3. Written answers in **markdown cells**, not code comments.

Every pattern you submit must be accompanied by (i) the count of what it accepted, (ii) the count of what it rejected, and (iii) a sentence on whether the rejections are defects or evidence that your pattern is wrong. A pattern with no rejection count is an incomplete answer.

Rubric

Part	Points	Full credit
1 Anatomy	15	The three functions distinguished <i>with output</i> ; the raw-string failure demonstrated, not asserted
2 Validation	20	Correct counts; <code>re.I</code> discussed as a policy change rather than a fix
3 Extraction	30	Named groups; the size field handles the missing-body case; non-IPv4 rows classified, not dropped
4 Substitution	15	Correct counts; unit incomparability raised without being prompted
5 pandas	10	Both shapes explained; the nullable-boolean fix applied and justified
6 Judgment	10	The looser pattern identified as worse; backtracking demonstrated by timing
Challenge	+10	A finding with its denominator

Use of AI tools

Parts 1–3 are to be completed **without AI assistance**. You are building the reflexes that make the rest of the course possible, and a pattern you did not write is a pattern you cannot debug at 2am.

For Parts 4–6 you may use an approved tool to compare approaches or to debug, provided you record, for each interaction: the exact prompt, a one-line summary of the response, and whether you accepted, modified, or rejected it, with a reason. Put the log in a markdown cell at the end of the notebook.

You remain responsible for every pattern you submit. “The tool wrote it” is not a defense for a regex that silently accepts `999.999.999.999`.

Appendix — regex quick reference

construct	means
<code>re.fullmatch(p, s)</code>	the whole string must match — use this to validate
<code>re.match(p, s)</code>	anchored at the start only; returns a truthy object for a mere prefix
<code>re.search(p, s)</code>	the first match anywhere
<code>re.findall(p, s)</code>	every match as a list of strings (or of tuples, if <code>p</code> has groups)
<code>re.finditer(p, s)</code>	every match as a match <i>object</i> — spans, named groups, lazy
<code>re.sub(p, r, s)</code>	replace; <code>r</code> may be a string or a function of the match
<code>re.split(p, s)</code>	split on a pattern
<code>re.compile(p, flags)</code>	compile once, give the pattern a name
<code>\d \s \w</code>	digit, whitespace, word character (Unicode unless <code>re.A</code>)
<code>\.</code>	a literal dot; unescaped <code>.</code> is <i>any</i> character
<code>\b</code>	a word boundary — and the reason raw strings matter
<code>[abc] [^abc]</code>	one of / not one of
<code>-</code> inside a class	a range unless escaped or placed first or last
<code>+ * ?</code>	one-or-more, zero-or-more, optional
<code>{4} {2,} {1,2}</code>	exact, at-least, and range counts
<code>+? *?</code>	lazy: take as little as possible
<code>() (?:)</code>	capture / group without capturing
<code>(?P<name>...)</code>	a named group; read it back with <code>groupdict()</code>
<code> </code>	alternation — the lowest precedence of any operator
<code>re.I</code>	ignore case
<code>re.X</code>	verbose: whitespace ignored, <code>#</code> comments allowed
<code>re.A</code>	restrict <code>\d \s \w</code> to ASCII
<code>re.M re.S</code>	<code>^\$</code> per line / <code>.</code> crosses newlines
<code>.str.fullmatch(p)</code>	the same question, one column at a time
<code>.str.extract(p)</code>	first match per row, as columns
<code>.str.extractall(p)</code>	every match, as a MultiIndexed frame
<code>.str.replace(p, r, regex=True)</code>	<code>re.sub</code> over a column